

Transport Layer

Lecture#13-18



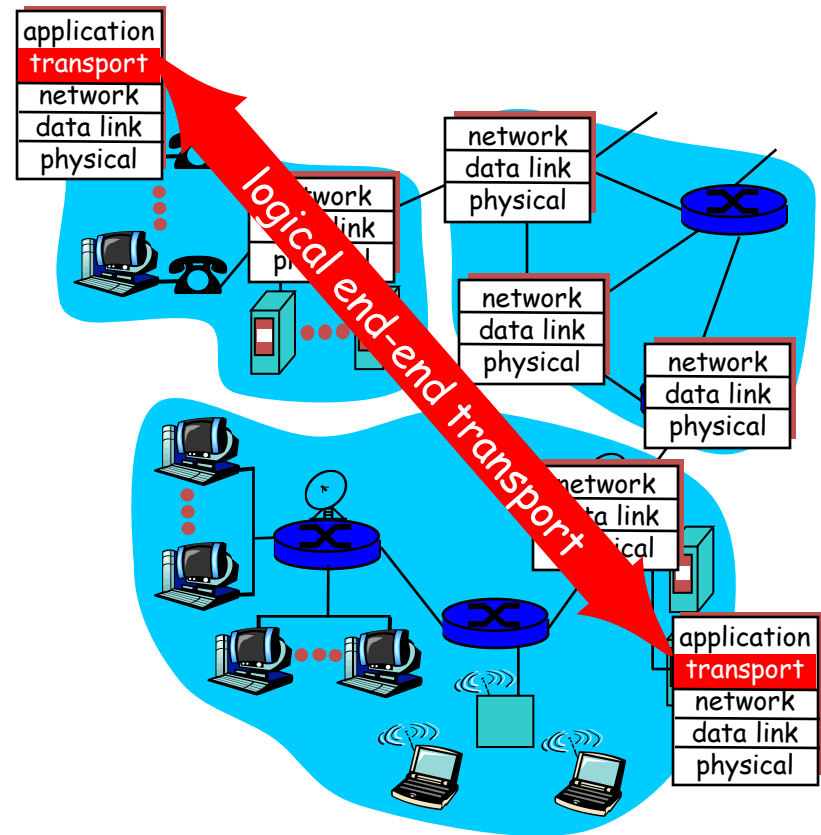
Dr. Sanjeev Patel
Asst Professor, CSE Dept.
NIT Rourkela

Outline

- Transport-layer services
- Connectionless transport: UDP
- Principles of reliable data transfer
- Connection-oriented transport: TCP
- TCP segment structure
- TCP RTT Calculation
- Connection management
- TCP congestion control

Transport services and protocols [6]

- provide *logical communication* between app processes running on different hosts
- transport protocols run in end systems
 - send side: breaks app messages into **segments**, passes to network layer
 - rcv side: reassembles segments into messages, passes to app layer
- more than one transport protocol available to apps
 - Internet: TCP and UDP



Transport vs. Network Layer [6]

- *network layer*: logical communication between hosts
- *transport layer*: logical communication between processes
 - relies on, enhances, network layer services

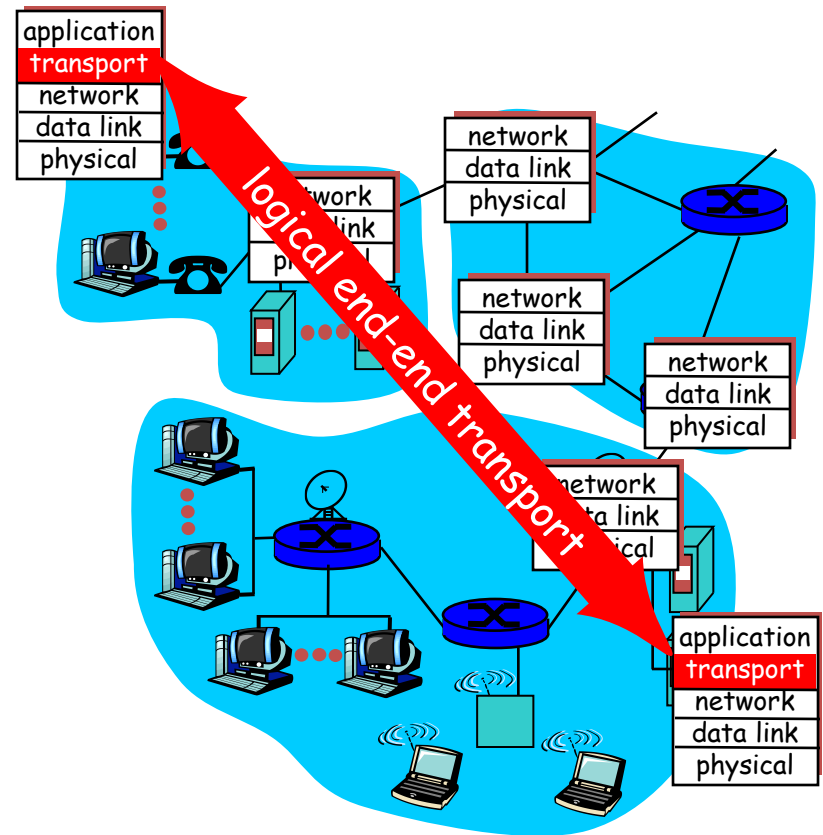
Household analogy:

12 kids sending letters to 12 kids

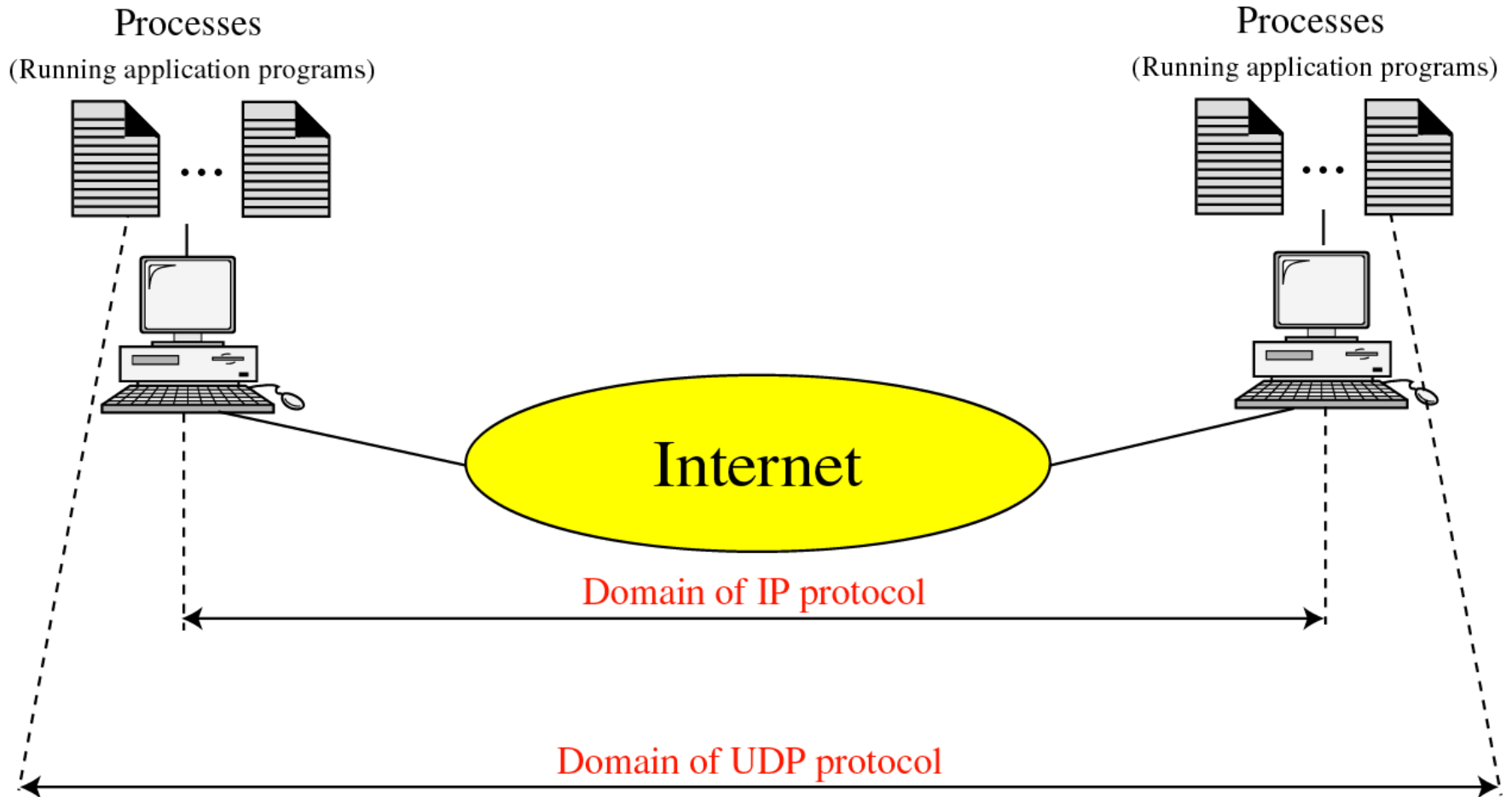
- processes = kids
- app messages = letters in envelopes
- hosts = houses
- transport protocol = Ann and Bill
- network-layer protocol = postal service

Internet transport-layer protocols [6]

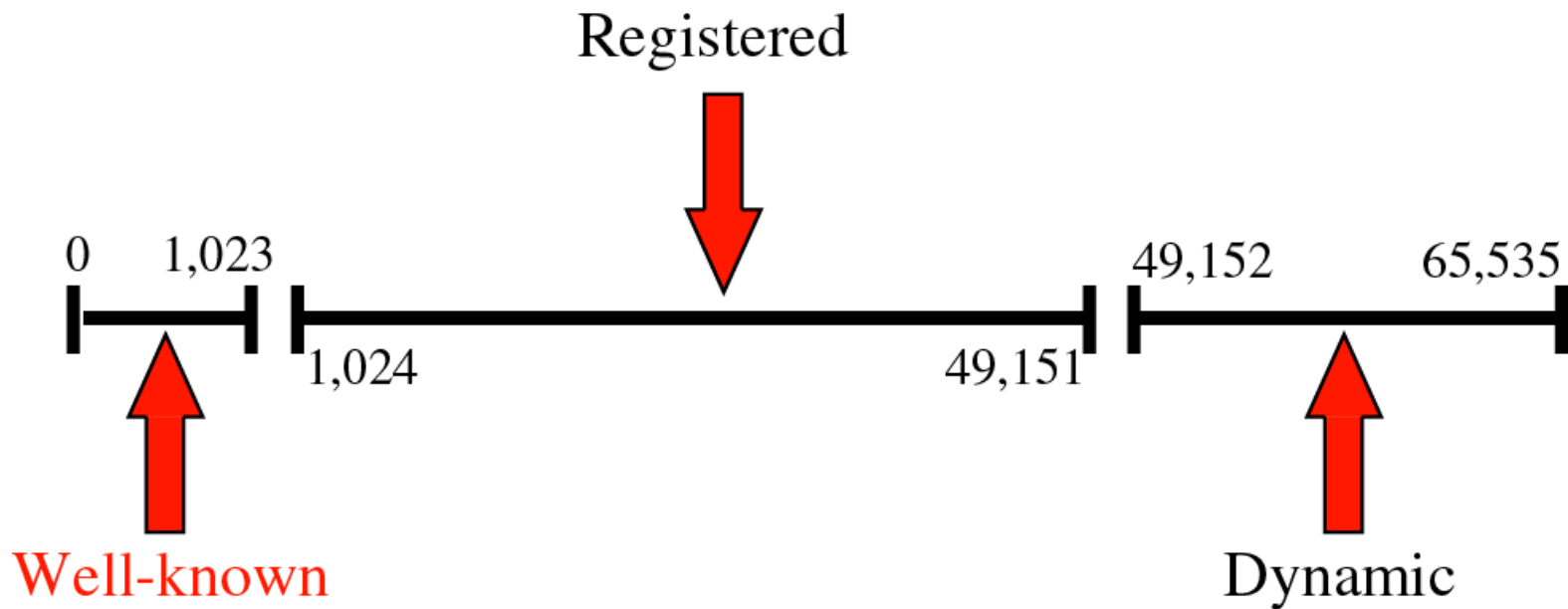
- reliable, in-order delivery (TCP)
 - congestion control
 - flow control
 - connection setup
- unreliable, unordered delivery: UDP
 - no-frills extension of “best-effort” IP
- services not available:
 - delay guarantees
 - bandwidth guarantees



UDP versus IP [5]



Internet Assigned Numbers Authority (IANA) ranges [5]



UDP: User Datagram Protocol [RFC 768]

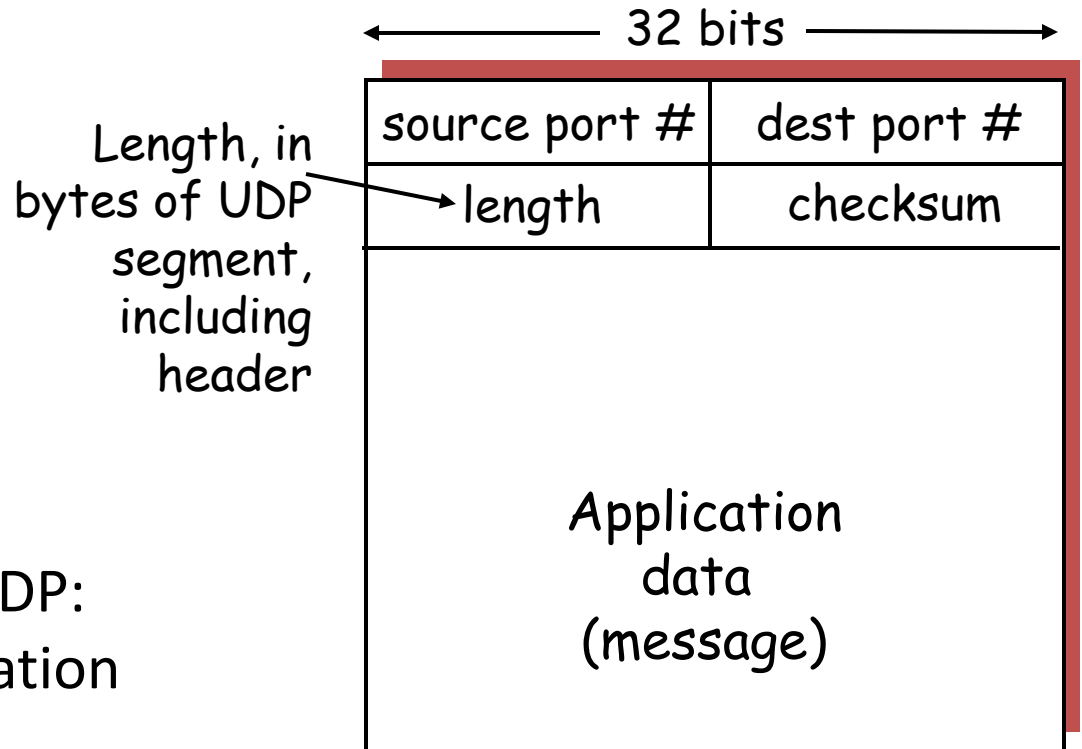
- “no frills,” “bare bones”
Internet transport protocol
- “best effort” service, UDP segments may be:
 - lost
 - delivered out of order to app
- *connectionless*:
 - no handshaking between UDP sender, receiver
 - each UDP segment handled independently of others

Why is there a UDP?

- no connection establishment (which can add delay)
- simple: no connection state at sender, receiver
- small segment header
- no congestion control: UDP can blast away as fast as desired

UDP: more [6]

- often used for streaming multimedia apps
 - loss tolerant
 - rate sensitive
- Protocols uses UDP
 - DNS
 - SNMP
- reliable transfer over UDP: add reliability at application layer
 - application-specific error recovery!



UDP segment format

UDP checksum

Goal: detect “errors” (e.g., flipped bits) in transmitted segment

Sender:

- treat segment contents as sequence of 16-bit integers
- checksum: addition (1’s complement sum) of segment contents
- sender puts checksum value into UDP checksum field

Receiver:

- compute checksum of received segment
- check if computed checksum equals checksum field value:
 - NO - error detected
 - YES - no error detected. *But maybe errors nonetheless?*
More later
 - No error, if sum is all 1s or 1’s complement is all 0s

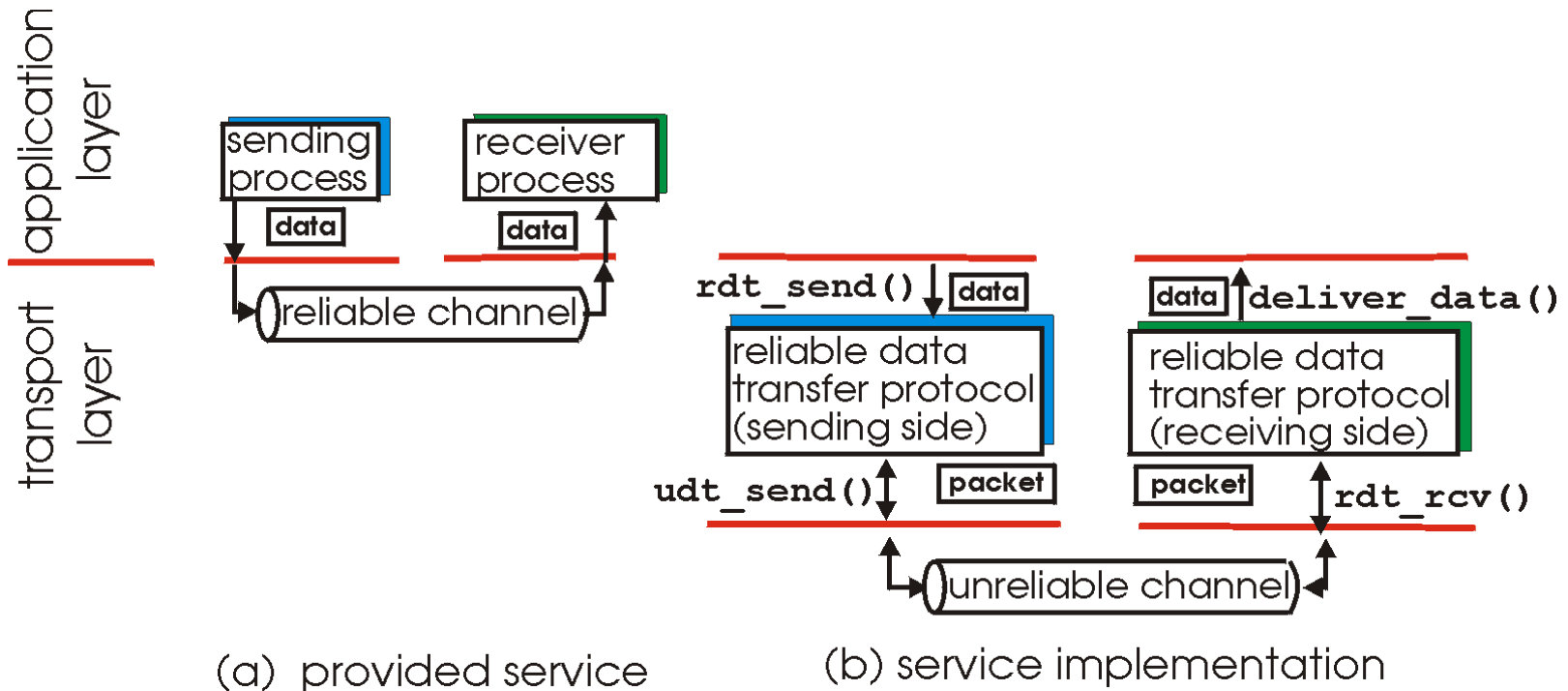
Internet Checksum Example

- Note
 - When adding numbers, a carryout from the most significant bit needs to be added to the result
- Example: add two 16-bit integers

	1	1	1	0	0	1	1	0	0	1	1	0	0	1	1	0
	1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
wraparound	1	1	0	1	1	1	0	1	1	1	0	1	1	1	0	1
sum	1	0	1	1	1	0	1	1	1	0	1	1	1	1	0	0
checksum	0	1	0	0	0	1	0	0	0	1	0	0	0	0	1	1

Principles of Reliable data transfer [6]

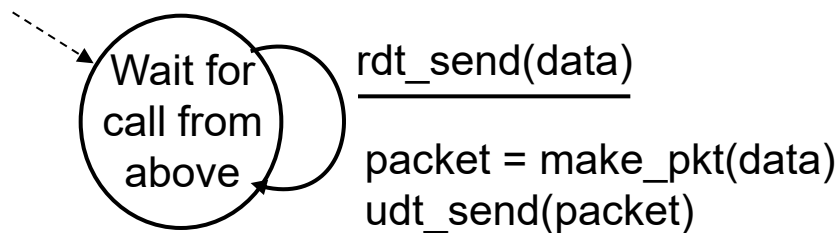
- important in app., transport, link layers
- top-10 list of important networking topics!



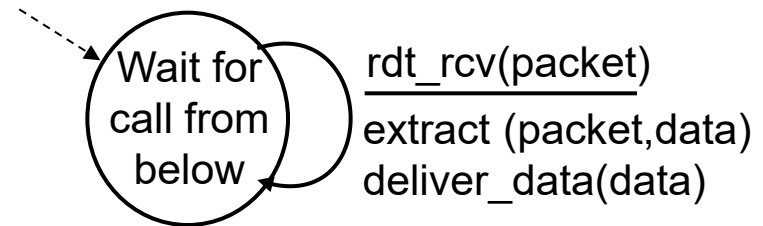
- characteristics of unreliable channel will determine complexity of reliable data transfer protocol (rdt)

Rdt1.0: reliable transfer over a reliable channel [6]

- underlying channel perfectly reliable
 - no bit errors
 - no loss of packets
- separate FSMs for sender, receiver:
 - sender sends data into underlying channel
 - receiver read data from underlying channel



sender



receiver

Rdt2.0: channel with bit errors [6]

- underlying channel may flip bits in packet
 - checksum to detect bit errors
- *the* question: how to recover from errors:
 - *acknowledgements (ACKs)*: receiver explicitly tells sender that pkt received OK
 - *negative acknowledgements (NAKs)*: receiver explicitly tells sender that pkt had errors
 - sender retransmits pkt on receipt of NAK
- new mechanisms in **rdt2.0** (beyond **rdt1.0**):
 - error detection
 - receiver feedback: control msgs (ACK,NAK) rcvr->sender

rdt2.0 has a fatal flaw! [6]

What happens if ACK/NAK corrupted?

- sender doesn't know what happened at receiver!
- can't just retransmit: possible duplicate

Handling duplicates:

- sender adds *sequence number* to each pkt
- sender retransmits current pkt if ACK/NAK garbled
- receiver discards (doesn't deliver up) duplicate pkt

stop and wait

Sender sends one packet, then waits for receiver response

rdt2.1: discussion

Sender:

- seq # added to pkt
- two seq. #'s (0,1) will suffice. Why?
- must check if received ACK/NAK corrupted
- twice as many states
 - state must “remember” whether “current” pkt has 0 or 1 seq. #

Receiver:

- must check if received packet is duplicate
 - state indicates whether 0 or 1 is expected pkt seq #
- note: receiver can *not* know if its last ACK/NAK received OK at sender

rdt2.2: a NAK-free protocol

- same functionality as rdt2.1, using ACKs only
- instead of NAK, receiver sends ACK for last pkt received OK
 - receiver must *explicitly* include seq # of pkt being ACKed
- duplicate ACK at sender results in same action as NAK: *retransmit current pkt*

rdt3.0: channels with errors *and* loss

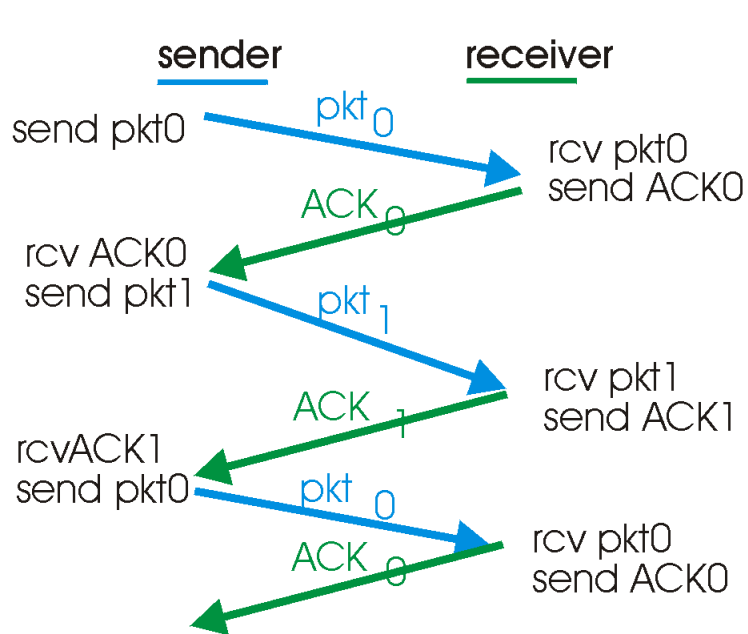
New assumption: underlying channel can also lose packets (data or ACKs)

- checksum, seq. #, ACKs, retransmissions will be of help, but not enough

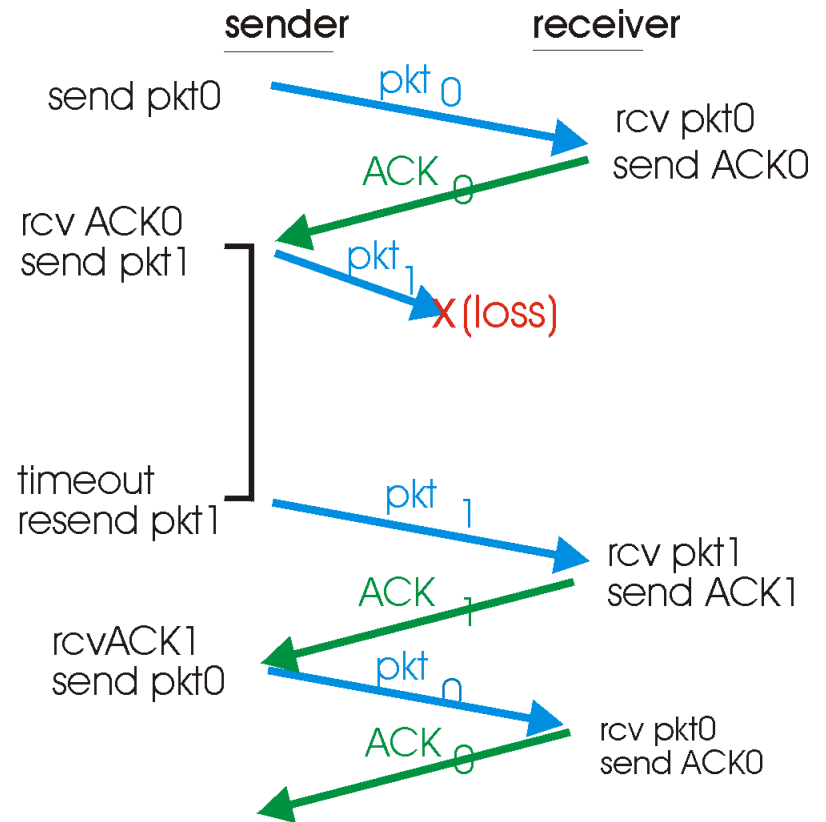
Approach: sender waits “reasonable” amount of time for ACK

- retransmits if no ACK received in this time
- if pkt (or ACK) just delayed (not lost):
 - retransmission will be duplicate, but use of seq. #'s already handles this
 - receiver must specify seq # of pkt being ACKed
- requires countdown timer

rdt3.0 in action

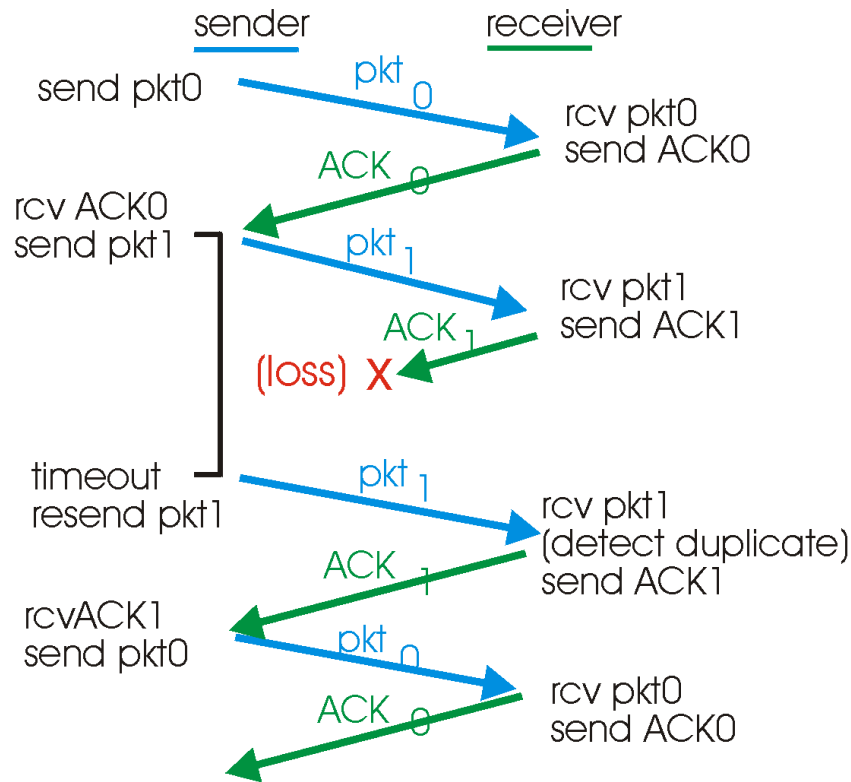


(a) operation with no loss

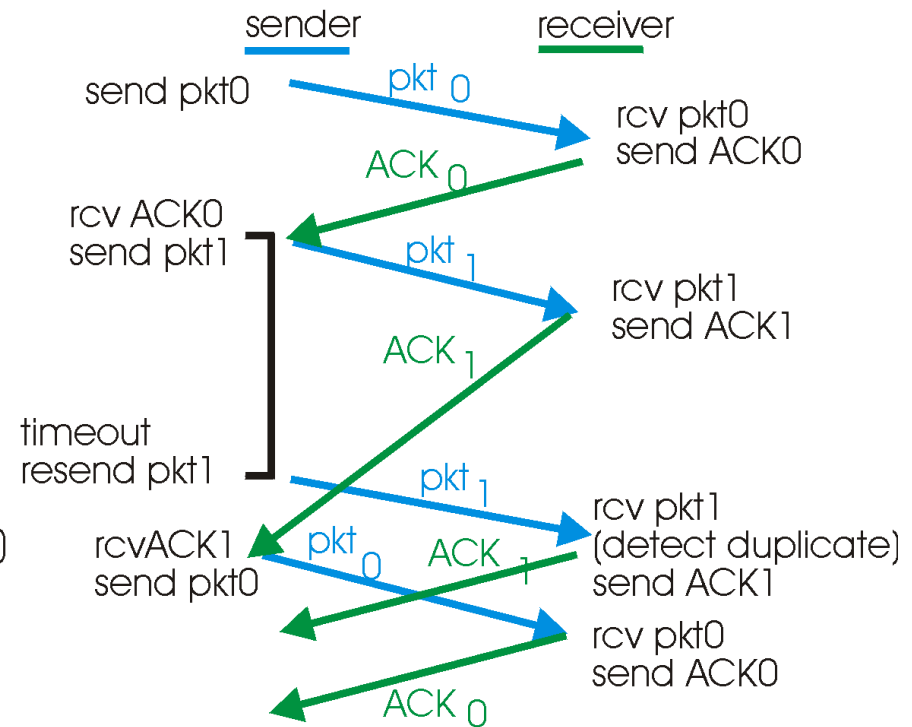


(b) lost packet

rdt3.0 in action



(c) lost ACK



(d) premature timeout

Performance of rdt3.0

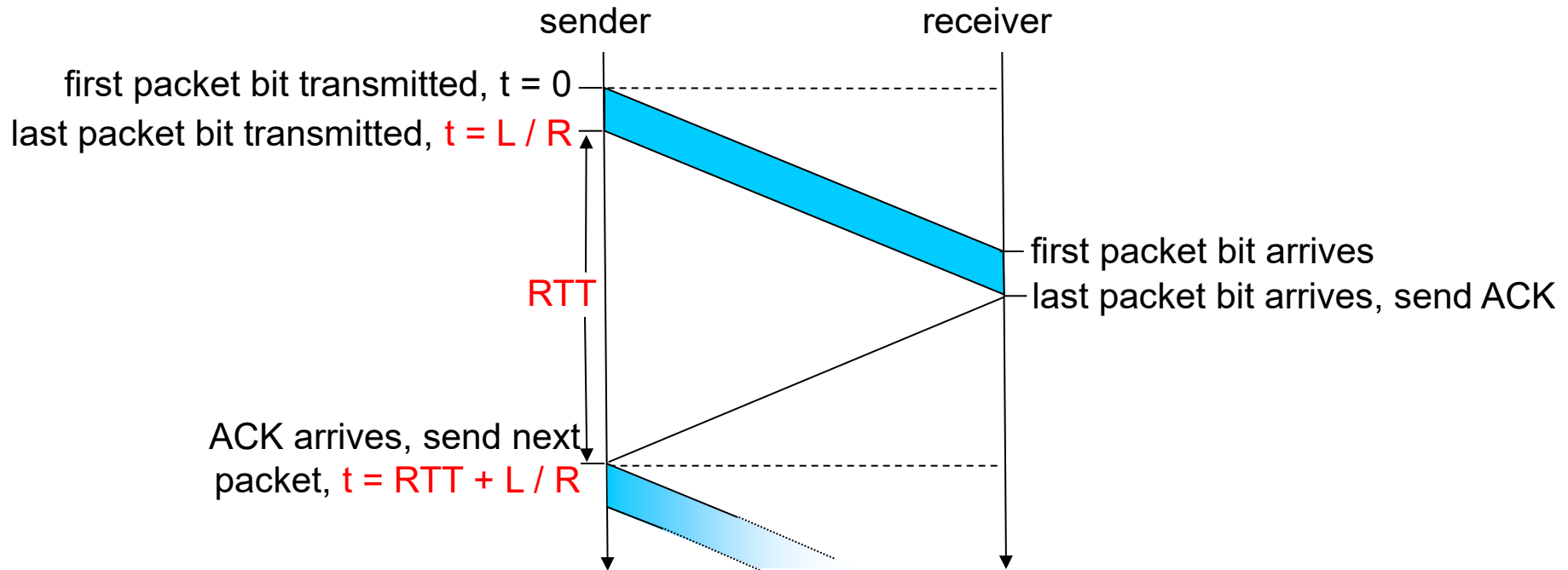
- rdt3.0 works, but performance stinks
- example: 1 Gbps link, 15 ms e-e prop. delay, 1KB packet:

$$T_{\text{transmit}} = \frac{L \text{ (packet length in bits)}}{R \text{ (transmission rate, bps)}} = \frac{8\text{kb/pkt}}{10^{**9} \text{ b/sec}} = 8 \text{ microsec}$$

$$U_{\text{sender}} = \frac{L / R}{RTT + L / R} = \frac{.008}{30.008} = 0.00027$$

- U_{sender} : **utilization** - fraction of time sender busy sending
- 1KB pkt every 30 msec -> 33kB/sec thruput over 1 Gbps link
- network protocol limits use of physical resources!

rdt3.0: stop-and-wait operation

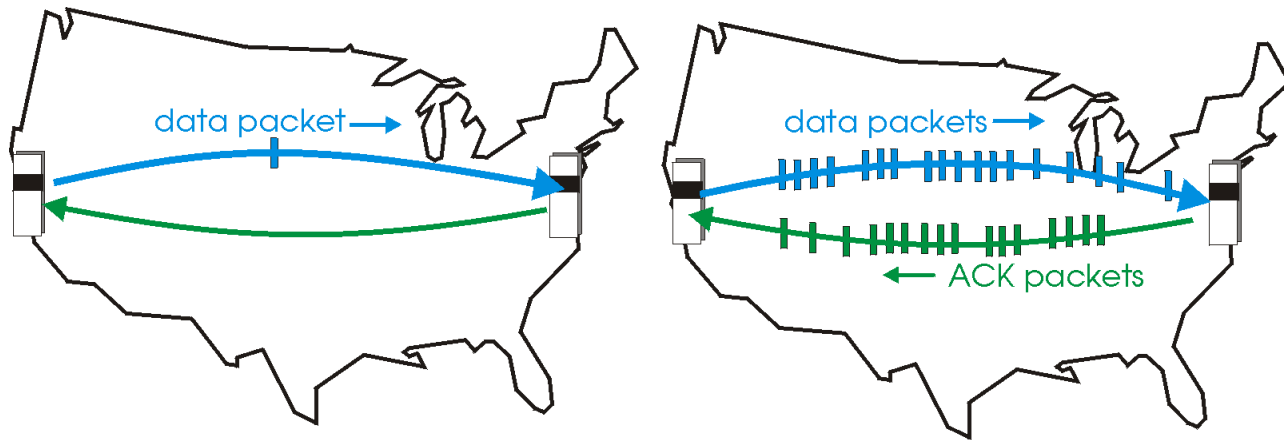


$$U_{\text{sender}} = \frac{L / R}{RTT + L / R} = \frac{.008}{30.008} = 0.00027$$

Pipelined protocols

Pipelining: sender allows multiple, “in-flight”, yet-to-be-acknowledged pkts

- range of sequence numbers must be increased
- buffering at sender and/or receiver

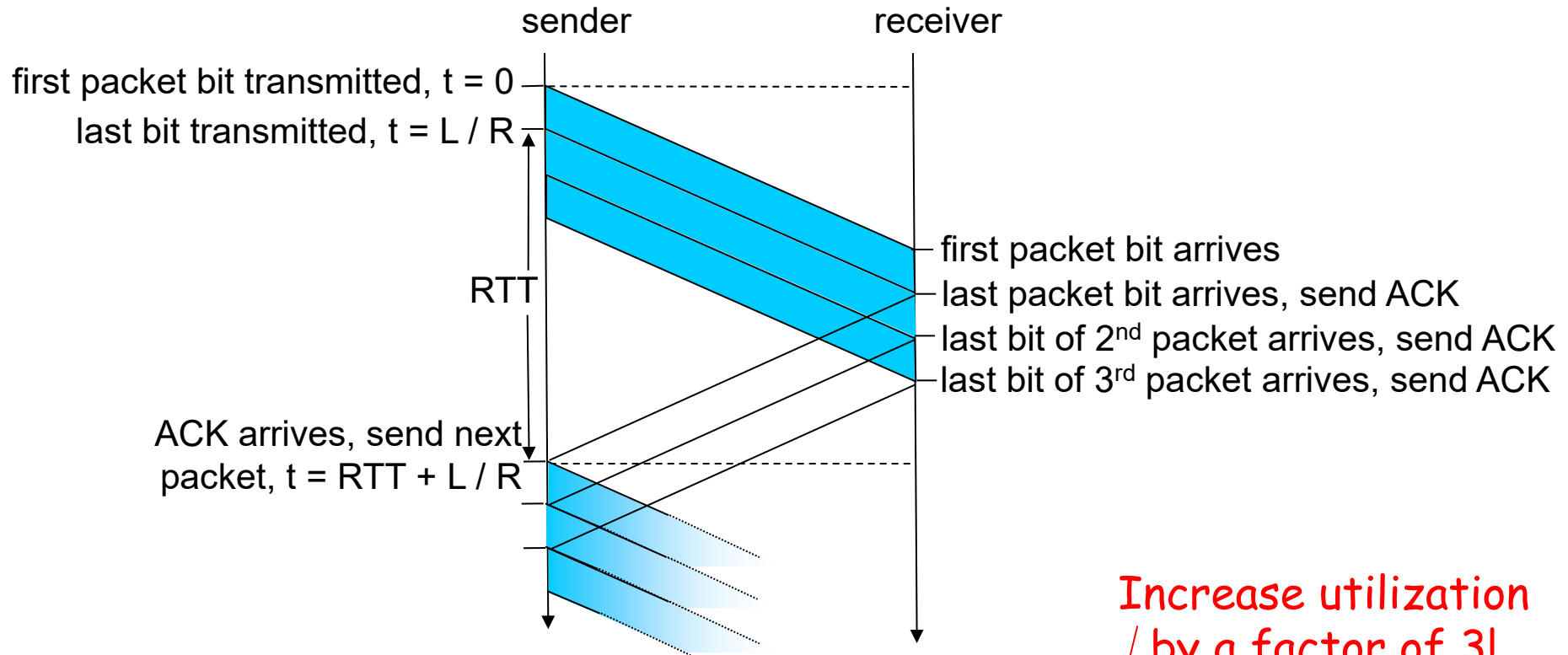


(a) a stop-and-wait protocol in operation

(b) a pipelined protocol in operation

- Two generic forms of pipelined protocols: *go-Back-N, selective repeat*

Pipelining: increased utilization



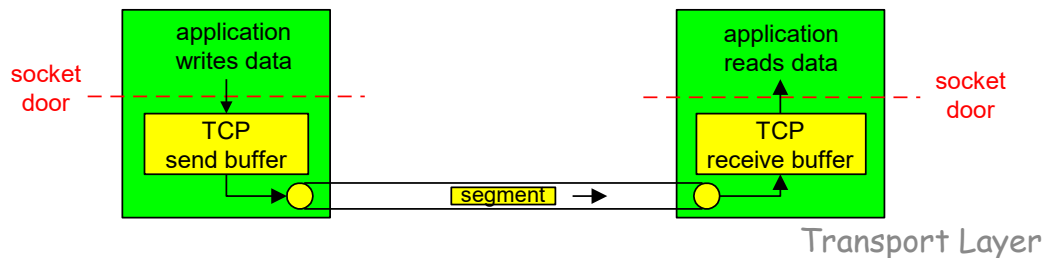
$$U_{\text{sender}} = \frac{3 * L / R}{RTT + L / R} = \frac{.024}{30.008} = 0.0008$$

Increase utilization
by a factor of 3!

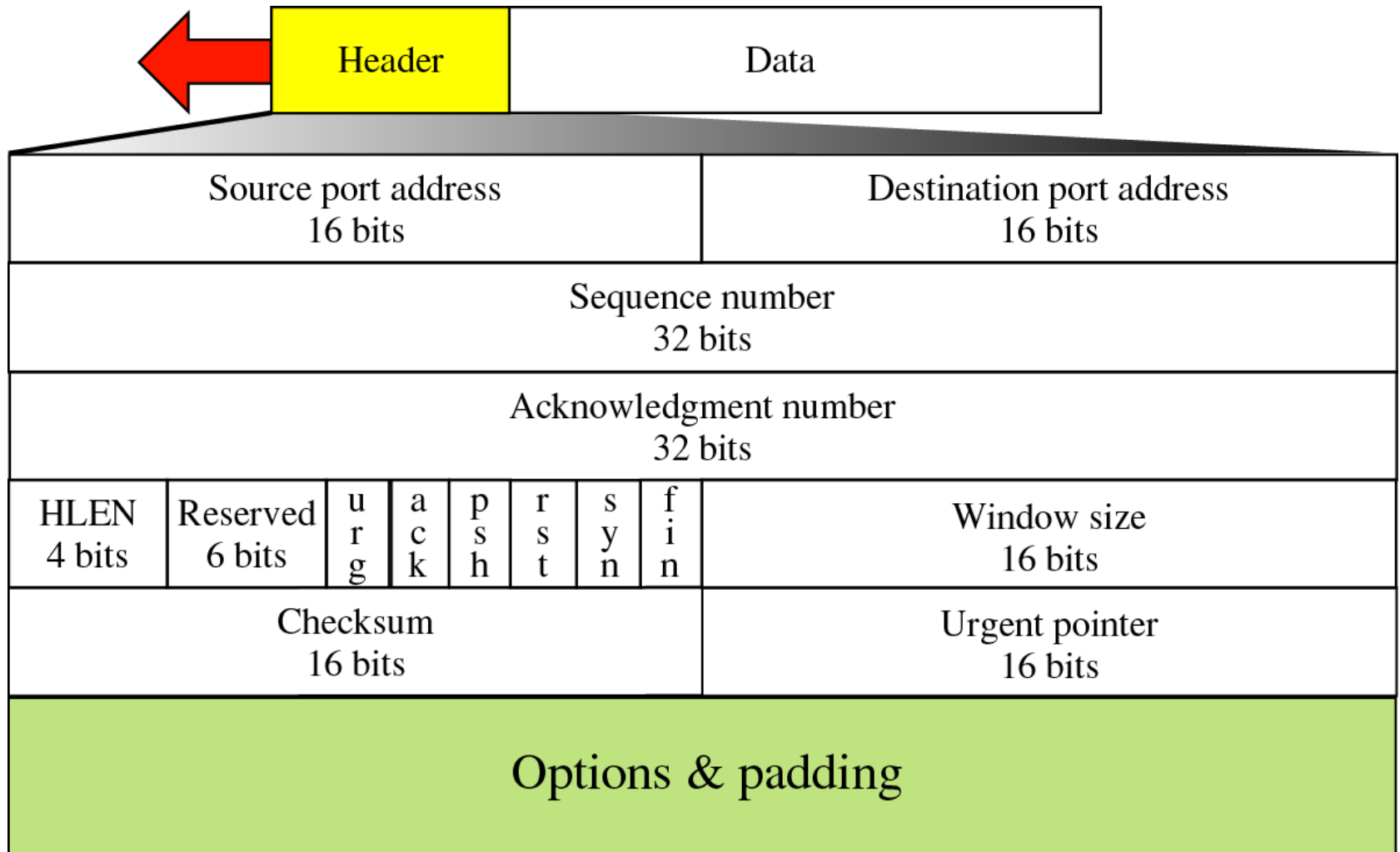
TCP: Overview

RFCs: 793, 1122, 1323, 2018, 2581

- **point-to-point:**
 - one sender, one receiver
- **reliable, in-order *byte stream*:**
 - no “message boundaries”
- **pipelined:**
 - TCP congestion and flow control set window size
- ***send & receive buffers***
- **full duplex data:**
 - bi-directional data flow in same connection
 - MSS: maximum segment size
- **connection-oriented:**
 - handshaking (exchange of control msgs) init's sender, receiver state before data exchange
- **flow controlled:**
 - sender will not overwhelm receiver



TCP segment format [5]



Control field [5]

URG: Urgent pointer is valid

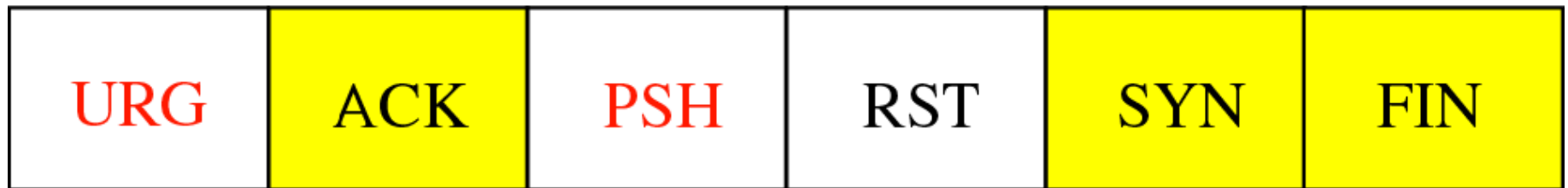
ACK: Acknowledgment is valid

PSH: Request for push

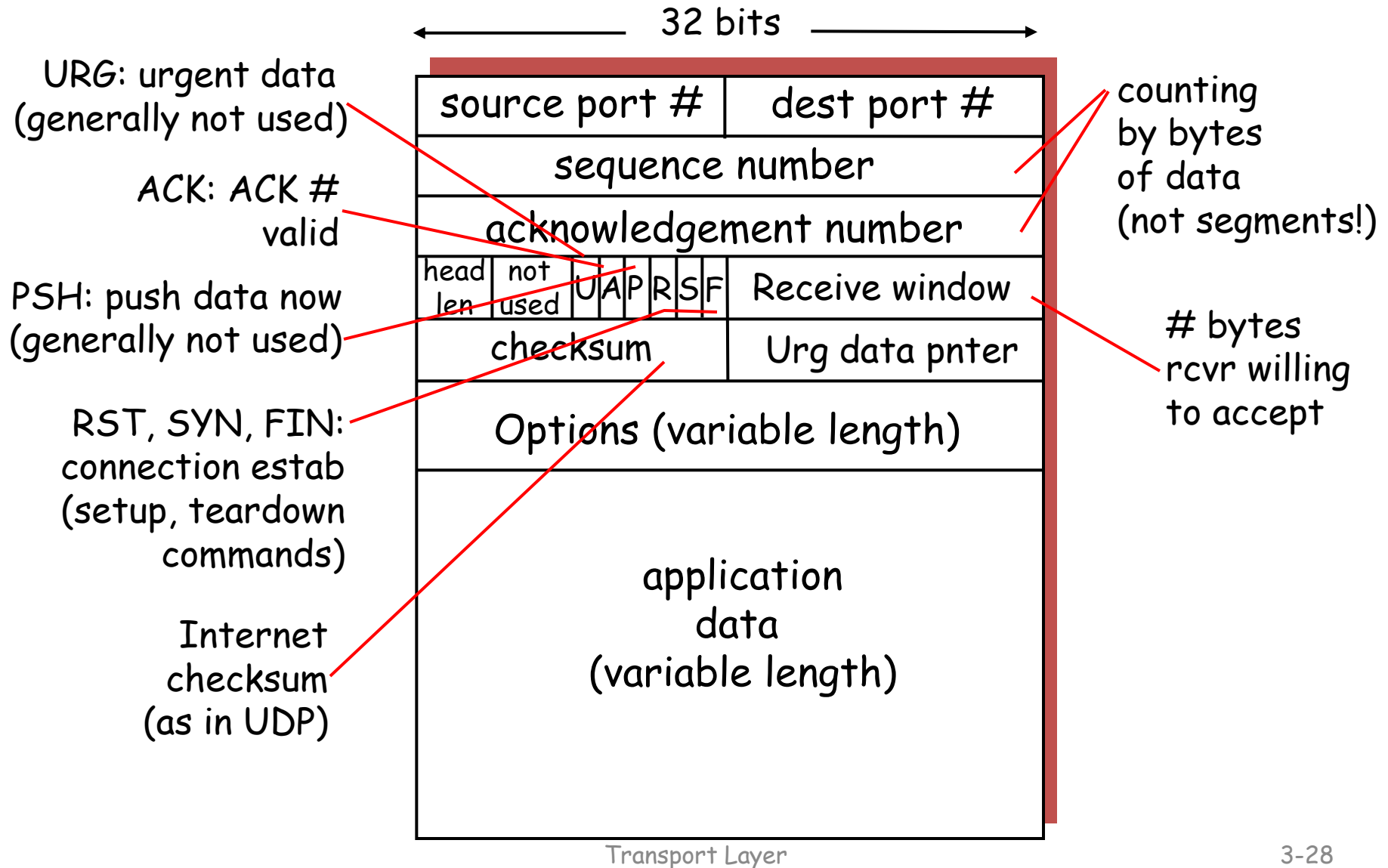
RST: Reset the connection

SYN: Synchronize sequence numbers

FIN: Terminate the connection



TCP segment structure [6]



TCP Round Trip Time and Timeout

Q: how to set TCP timeout value?

- longer than RTT
 - but RTT varies
- too short: premature timeout
 - unnecessary retransmissions
- too long: slow reaction to segment loss

Q: how to estimate RTT?

- **SampleRTT**: measured time from segment transmission until ACK receipt
 - ignore retransmissions
- **SampleRTT** will vary, want estimated RTT “smoother”
 - average several recent measurements, not just current **SampleRTT**

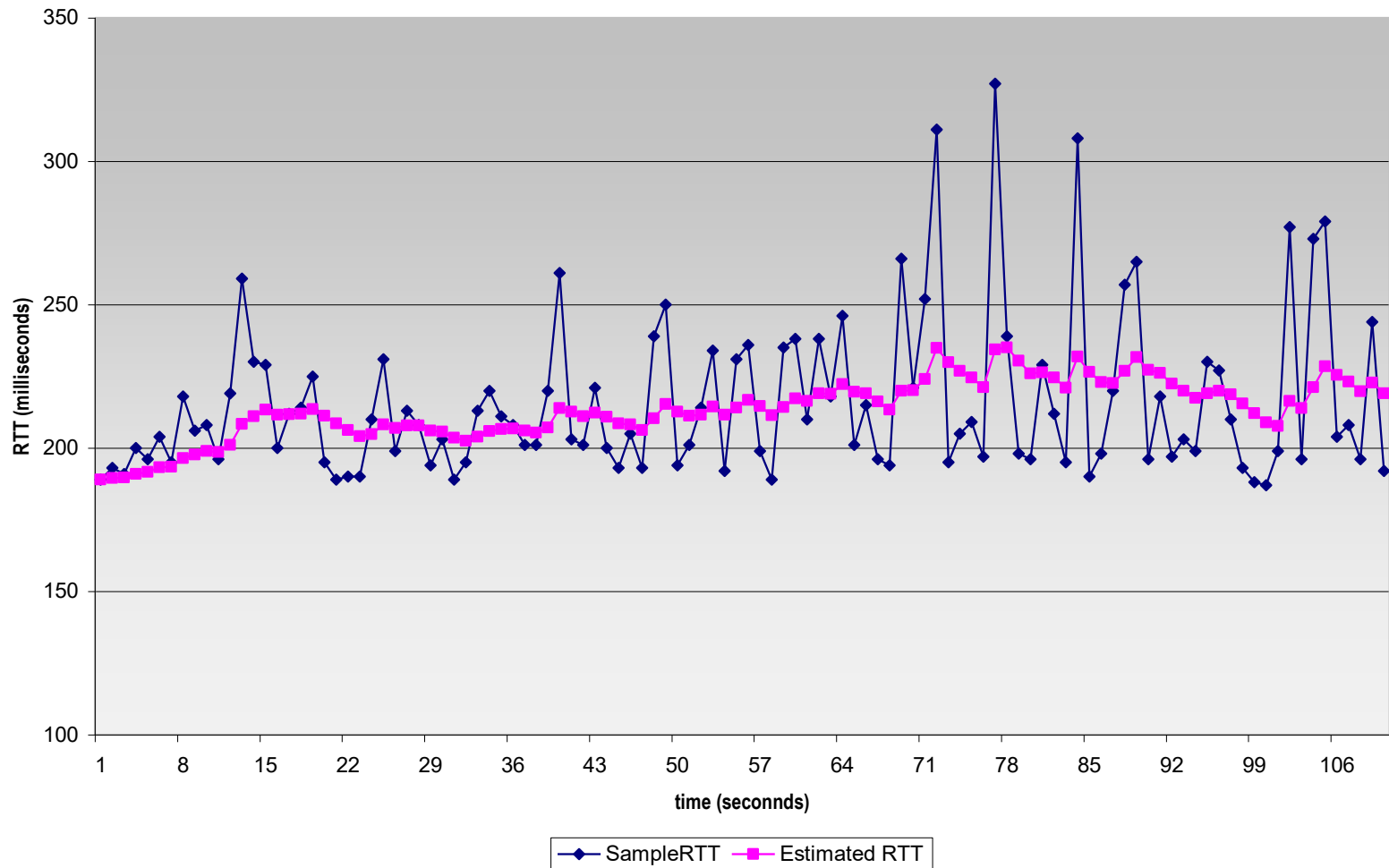
TCP Round Trip Time and Timeout

$$\text{EstimatedRTT} = (1 - \alpha) * \text{EstimatedRTT} + \alpha * \text{SampleRTT}$$

- ❑ Exponential weighted moving average
- ❑ influence of past sample decreases exponentially fast
- ❑ typical value: $\alpha = 0.125$

Example RTT estimation:

RTT: gaia.cs.umass.edu to fantasia.eurecom.fr



TCP Round Trip Time and Timeout

Setting the timeout

- **EstimatedRTT** plus “safety margin”
 - large variation in **EstimatedRTT** -> larger safety margin
- first estimate of how much **SampleRTT** deviates from **EstimatedRTT**:

$$\text{DevRTT} = (1-\beta) * \text{DevRTT} + \beta * |\text{SampleRTT} - \text{EstimatedRTT}|$$

(typically, $\beta = 0.25$)

Then set timeout interval:

$$\text{TimeoutInterval} = \text{EstimatedRTT} + 4 * \text{DevRTT}$$

RTO calculation for Time-out (Karn's Algorithm)

- Do not consider the round-trip time of a retransmitted segment in the calculation of RTTs.
- Do not update the value of RTTs until you send a segment and receive an acknowledgment without the need for retransmission.
- TCP does not consider the RTT of a retransmitted segment in its calculation of a new RTO.
- **Exponential Backoff Algorithm:** The value of RTO is doubled for each retransmission.
- So if the segment is retransmitted once, the value is two times the RTO. If it is transmitted twice, the value is four times the RTO, and so on.

TCP Connection Management

Recall: TCP sender, receiver
establish “connection” before
exchanging data segments

- initialize TCP variables:
 - seq. #s
 - buffers, flow control info
(e.g. **RcvWindow**)

- *client*: connection initiator

```
Socket clientSocket = new  
Socket("hostname", "port  
number");
```

- *server*: contacted by client

```
Socket connectionSocket =  
welcomeSocket.accept();
```

Three way handshake:

Step 1: client host sends TCP SYN
segment to server

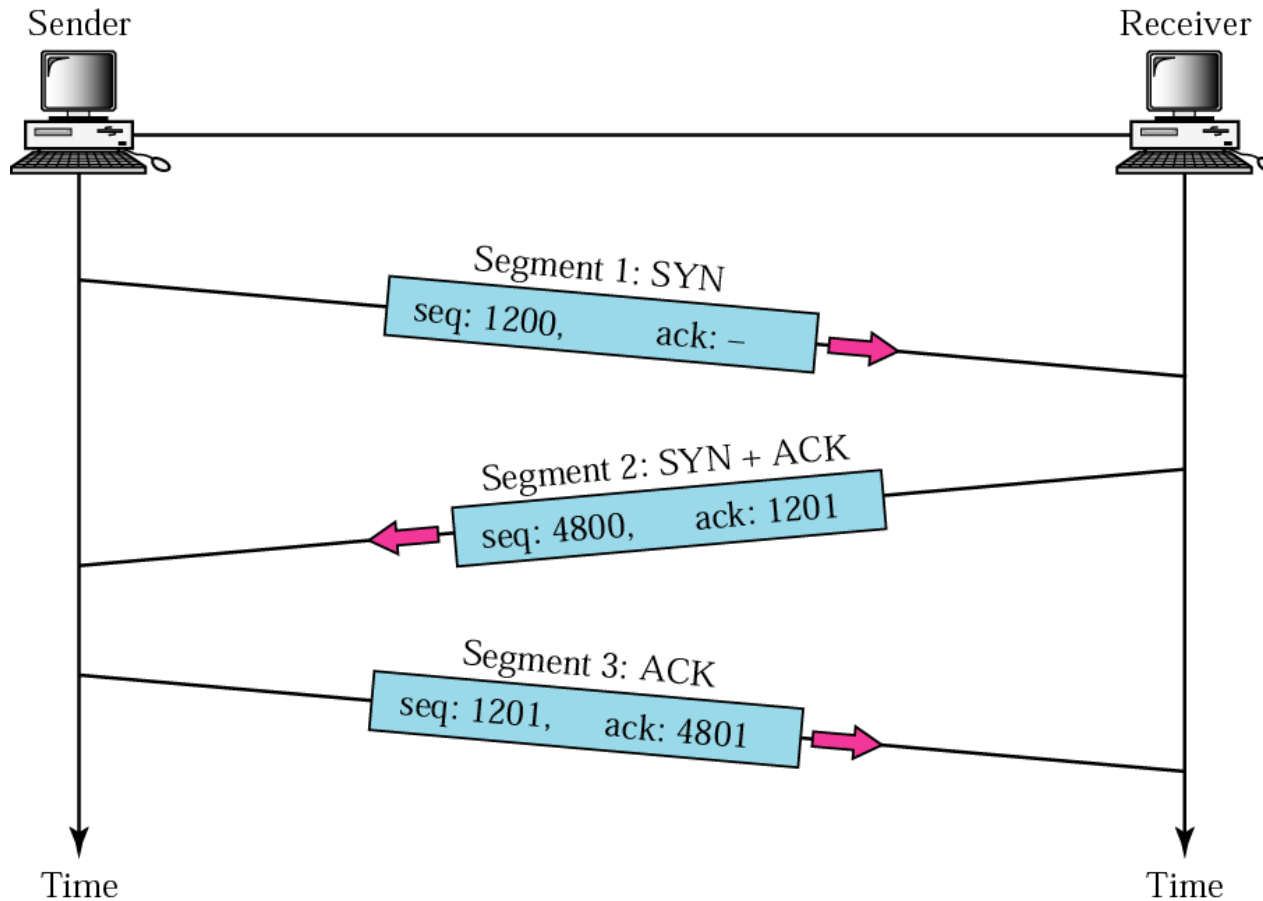
- specifies initial seq #
- no data

Step 2: server host receives SYN,
replies with SYNACK segment

- server allocates buffers
- specifies server initial seq. #

Step 3: client receives SYNACK, replies
with ACK segment, which may
contain data

Three-way handshake[5]



TCP Connection Management (cont.) [6]

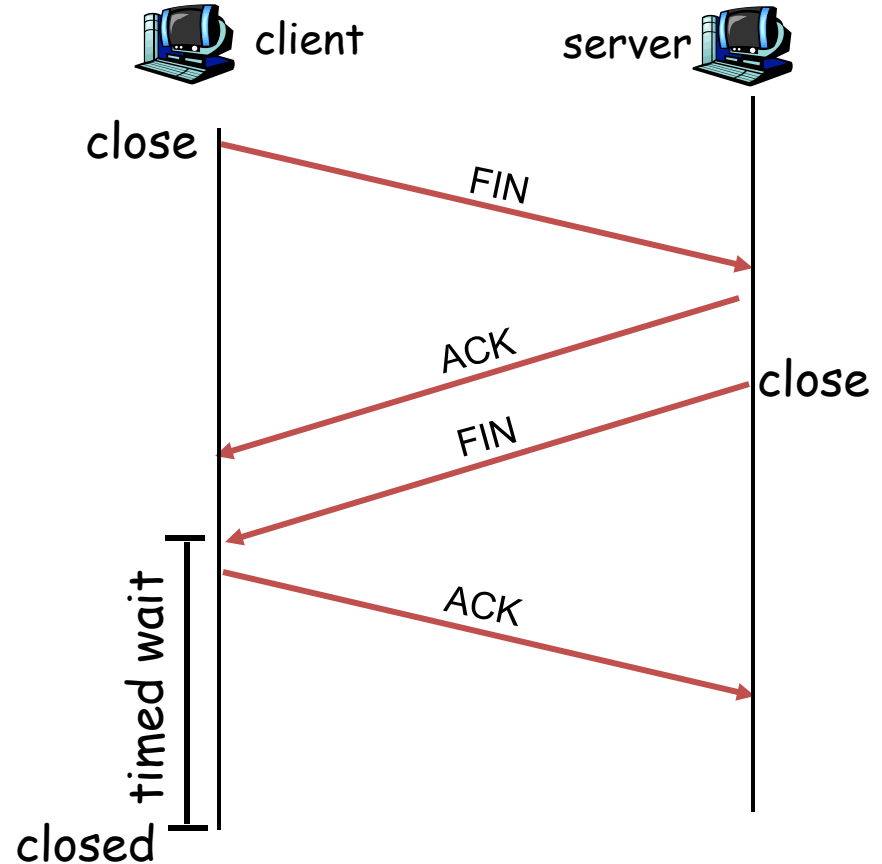
Closing a connection:

client closes socket:

```
clientSocket.close();
```

Step 1: client end system sends TCP FIN control segment to server

Step 2: server receives FIN, replies with ACK. Closes connection, sends FIN.



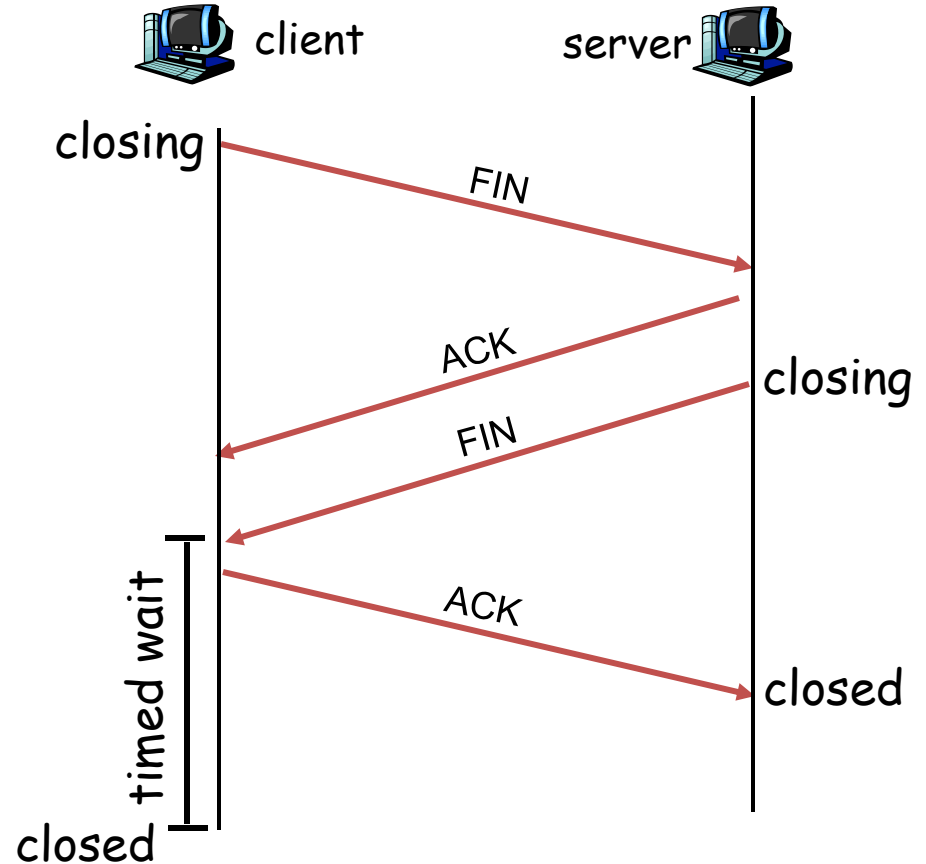
TCP Connection Management (cont.) [6]

Step 3: client receives FIN,
replies with ACK.

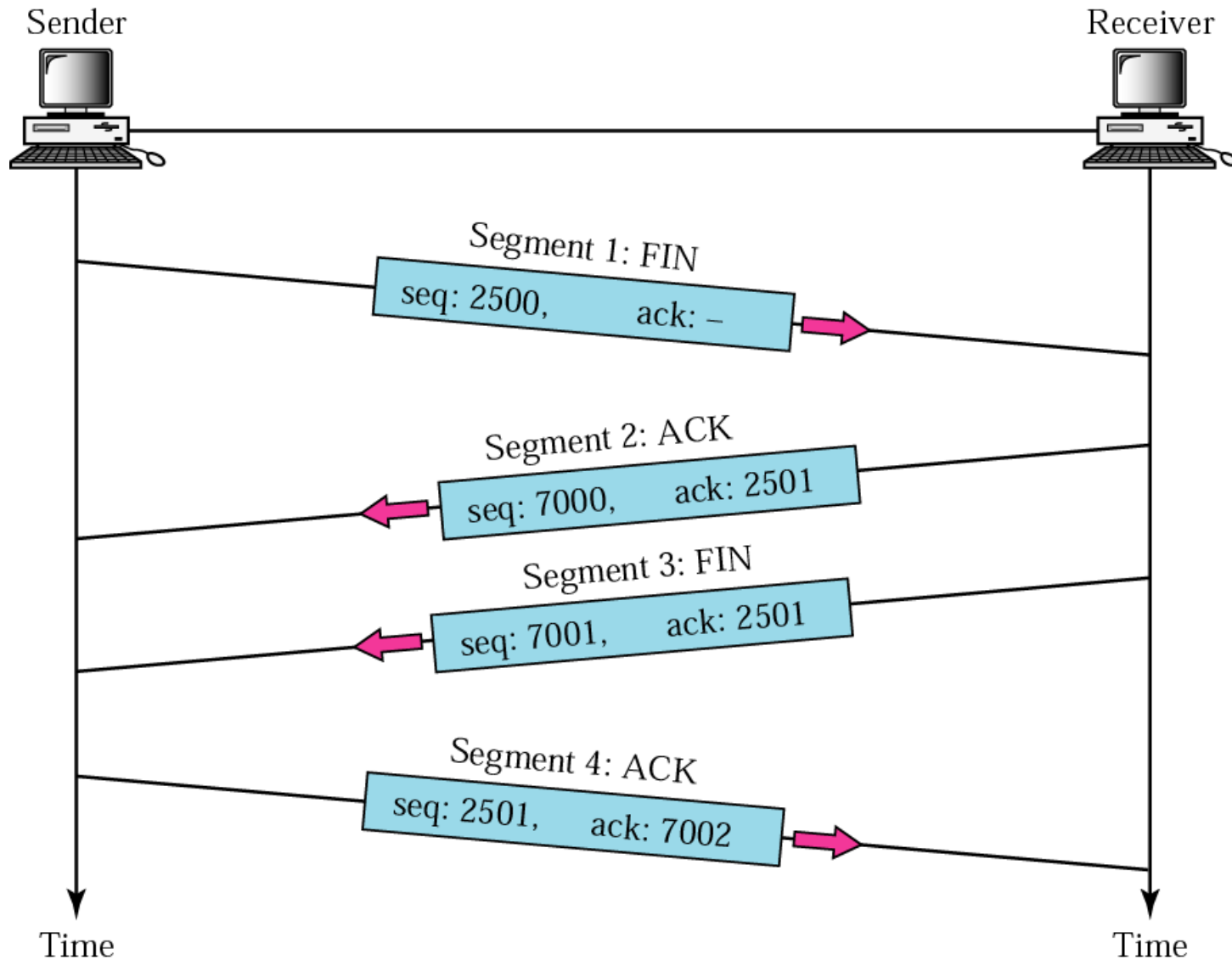
- Enters “timed wait” - will respond with ACK to received FINs

Step 4: server, receives ACK.
Connection closed.

Note: with small modification,
can handle simultaneous FINs.



Four-way handshake [5]



Principles of Congestion Control

Congestion:

- informally: “too many sources sending too much data too fast for *network* to handle”
- different from flow control!
- manifestations:
 - lost packets (buffer overflow at routers)
 - long delays (queueing in router buffers)
- a top-10 problem!

Approaches towards congestion control

Two broad approaches towards congestion control:

End-end congestion control:

- no explicit feedback from network
- congestion inferred from end-system observed loss, delay
- approach taken by TCP

Network-assisted congestion control:

- routers provide feedback to end systems
 - single bit indicating congestion (SNA, DECbit, TCP/IP ECN, ATM)
 - explicit rate sender should send at outgoing link

Case study: ATM ABR congestion control

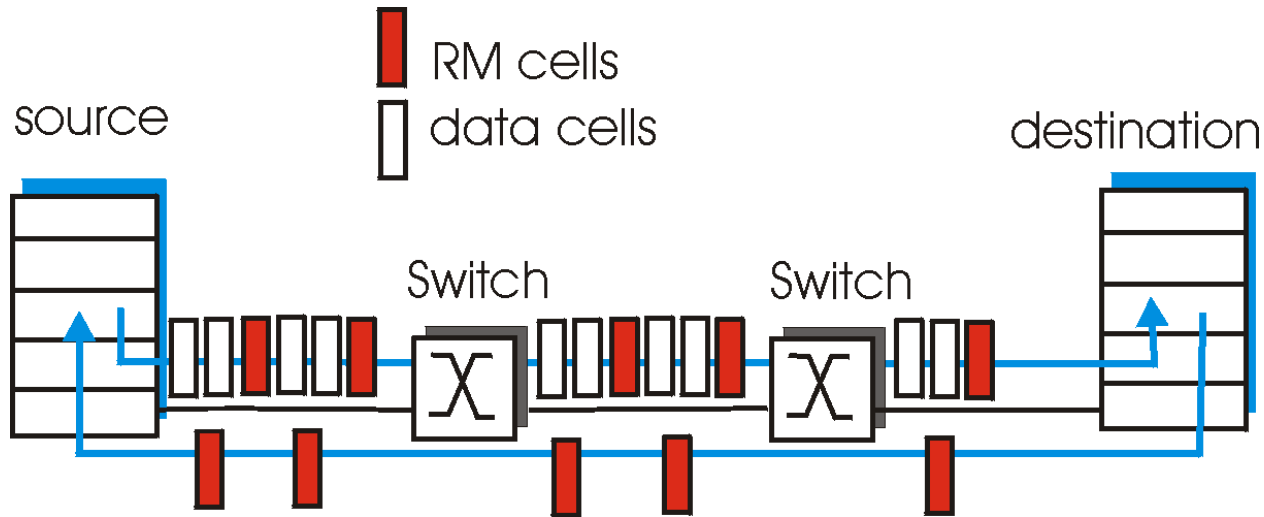
ABR: available bit rate:

- “elastic service”
- if sender’s path “underloaded”:
 - sender should use available bandwidth
- if sender’s path congested:
 - sender throttled to minimum guaranteed rate

RM (resource management) cells:

- sent by sender, interspersed with data cells
- bits in RM cell set by switches (“*network-assisted*”)
 - **NI bit**: no increase in rate (mild congestion)
 - **CI bit**: congestion indication
- RM cells returned to sender by receiver, with bits intact

Case study: ATM ABR congestion control [6]



- two-byte ER (explicit rate) field in RM cell
 - congested switch may lower ER value in cell
 - sender's send rate thus minimum supportable rate on path
- EFCI bit in data cells: set to 1 in congested switch
 - if data cell preceding RM cell has EFCI set to 1, receiver sets CI bit of RM cell to 1 and send back to sender.

TCP Congestion Control

- end-end control (no network assistance)
- sender limits transmission:
LastByteSent - LastByteAcked
≤ CongWin
- Roughly,

$$\text{rate} = \frac{\text{CongWin}}{\text{RTT}} \text{ Bytes/sec}$$

- **CongWin** is dynamic, function of perceived network congestion

How does sender perceive congestion?

- loss event = timeout *or* 3 duplicate acks
- TCP sender reduces rate (**CongWin**) after loss event

three mechanisms:

- AIMD
- slow start
- conservative after timeout events

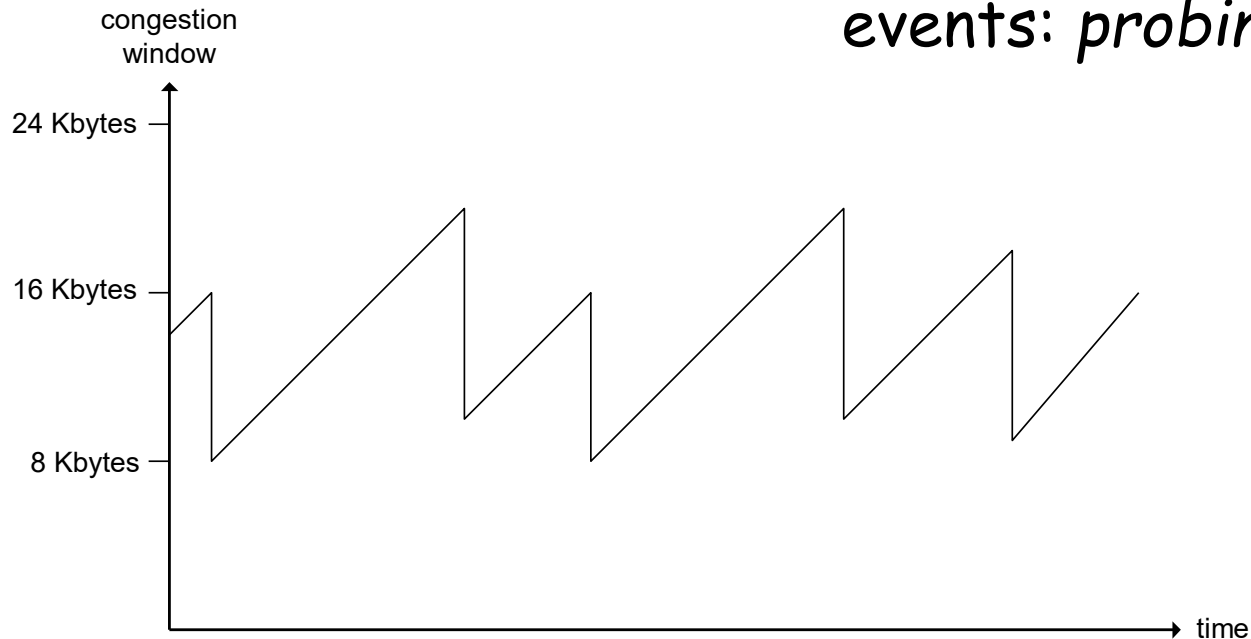
TCP AIMD [6]

multiplicative decrease: cut

CongWin in half after
loss event

additive increase:

increase **CongWin** by
1 MSS every RTT in
the absence of loss
events: *probing*



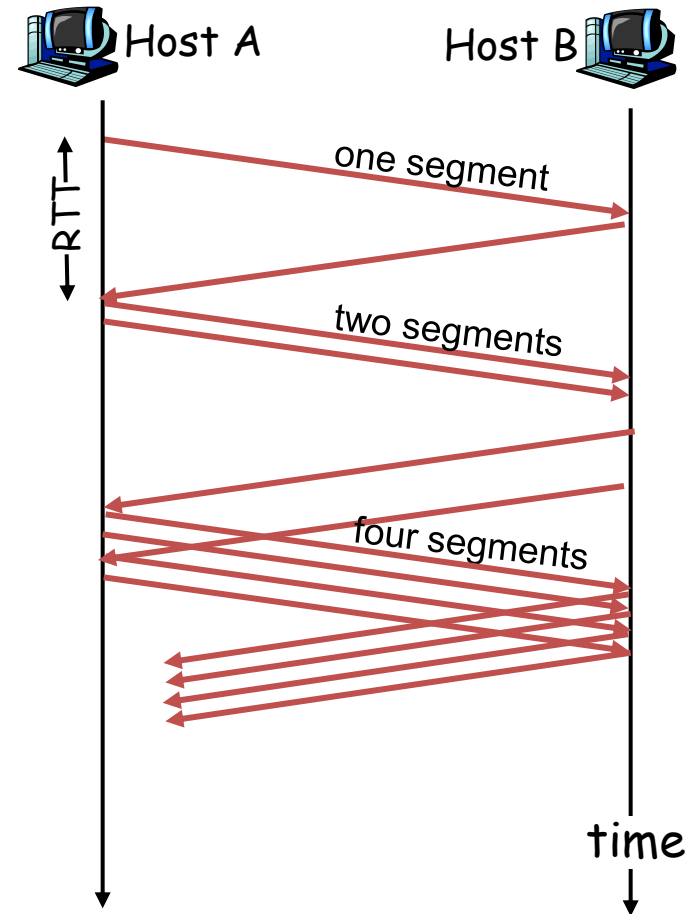
Long-lived TCP connection

TCP Slow Start

- When connection begins,
CongWin = 1 MSS
 - Example: MSS = 500 bytes & RTT = 200 msec
 - initial rate = 20 kbps
 - available bandwidth may be
>> MSS/RTT
 - desirable to quickly ramp up to respectable rate
- ❑ When connection begins, increase rate exponentially fast until first loss event

TCP Slow Start (more) [6]

- When connection begins, increase rate exponentially until first loss event:
 - double **CongWin** every RTT
 - done by incrementing **CongWin** for every ACK received
- Summary: initial rate is slow but ramps up exponentially fast



Refinement

- After 3 dup ACKs:
 - **CongWin** is cut in half
 - window then grows linearly
- But after timeout event:
 - **CongWin** instead set to 1 MSS;
 - window then grows exponentially
 - to a threshold, then grows linearly

Philosophy:

- 3 dup ACKs indicates network capable of delivering some segments
- timeout before 3 dup ACKs is "more alarming"

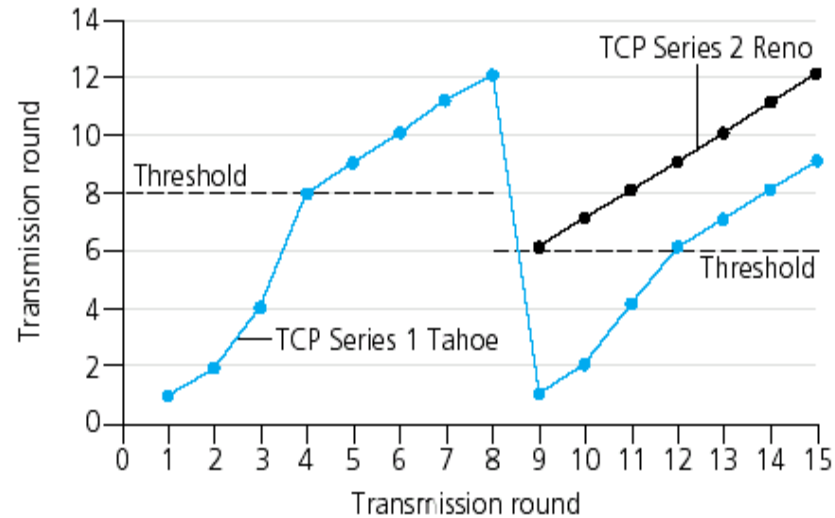
Refinement (more) [6]

Q: When should the exponential increase switch to linear?

A: When **CongWin** gets to 1/2 of its value before timeout.

Implementation:

- Variable Threshold
- At loss event, Threshold is set to 1/2 of CongWin just before loss event



TCP Throughput [6]

- What's the average throughput of TCP as a function of window size and RTT?
 - Ignore slow start
- Let W be the window size when loss occurs.
- When window is W , throughput is W/RTT
- Just after loss, window drops to $W/2$, throughput to $W/2RTT$.
- Average throughput: $.75 W/RTT$

TCP Futures

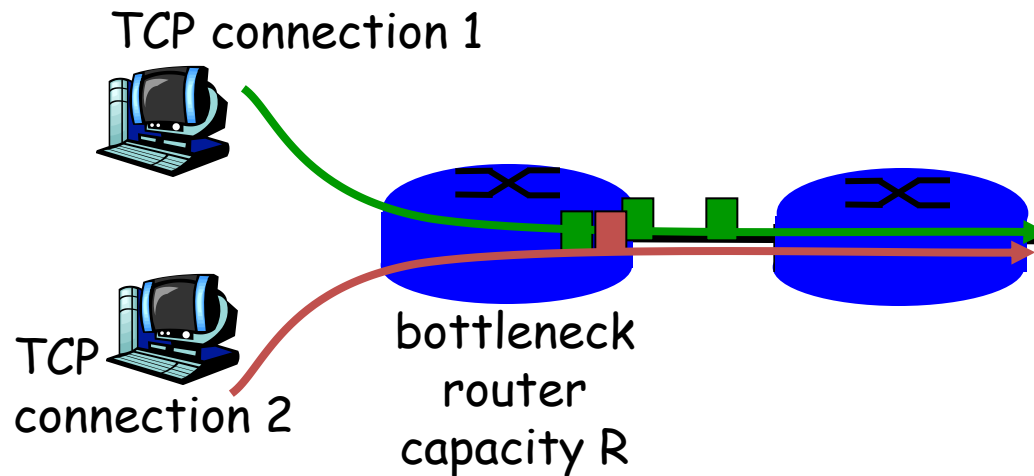
- Example: 1500 byte segments, 100ms RTT, want 10 Gbps throughput
- Requires window size $W = 83,333$ in-flight segments
- Throughput in terms of loss rate:

- $\rightarrow L = 2 \cdot 10^{-10}$ *Wow*
$$\frac{1.22 \cdot MSS}{RTT \sqrt{L}}$$

- New versions of TCP for high-speed needed!

TCP Fairness [6]

Fairness goal: if K TCP sessions share same bottleneck link of bandwidth R , each should have average rate of R/K



Fairness (more)

Fairness and UDP

- Multimedia apps often do not use TCP
 - do not want rate throttled by congestion control
- Instead use UDP:
 - pump audio/video at constant rate, tolerate packet loss
- Research area: TCP friendly

Fairness and parallel TCP connections

- nothing prevents app from opening parallel connections between 2 hosts.
- Web browsers do this
- Example: link of rate R supporting 9 connections;
 - new app asks for 1 TCP, gets rate $R/10$
 - new app asks for 11 TCPs, gets $R/2$!

Delay modeling [6]

Q: How long does it take to receive an object from a Web server after sending a request?

Ignoring congestion, delay is influenced by:

- TCP connection establishment
- data transmission delay
- slow start

Notation, assumptions:

- Assume one link between client and server of rate R
- S : MSS (bits)
- O : object size (bits)
- no retransmissions (no loss, no corruption)

Window size:

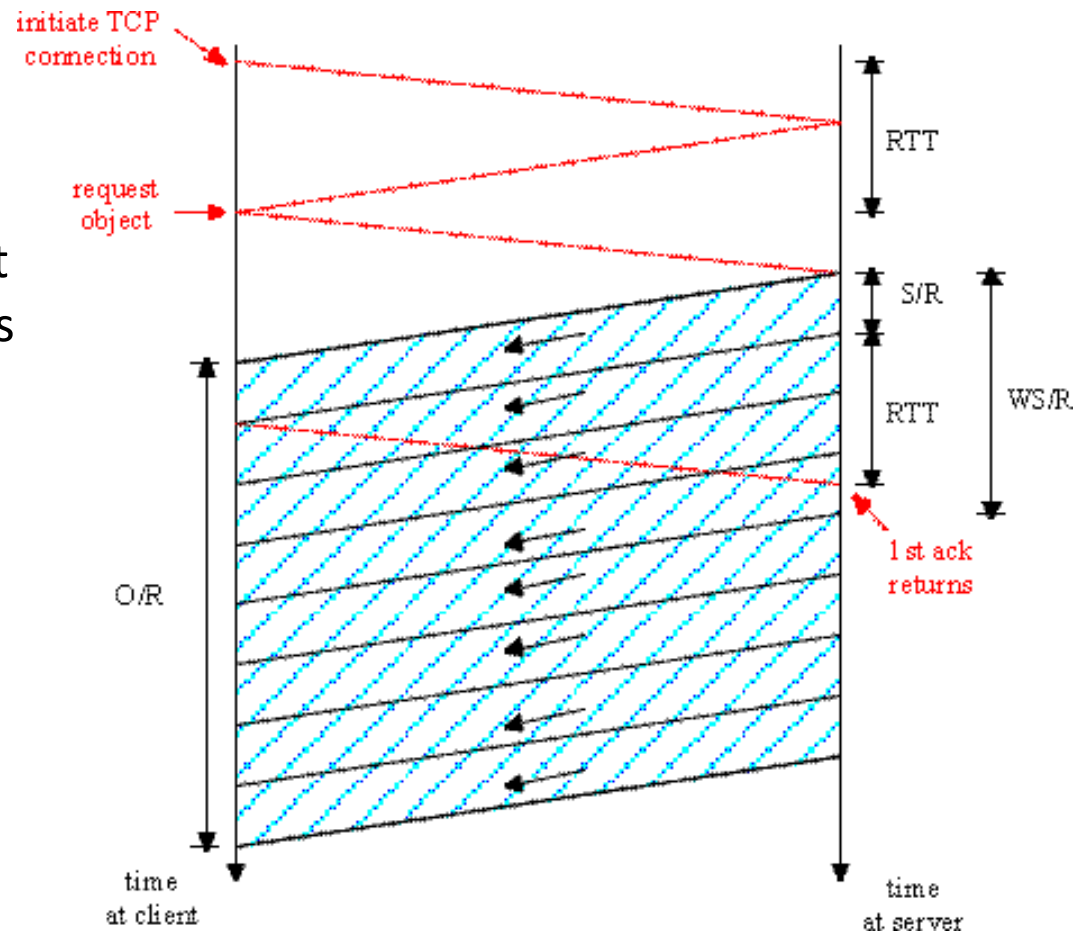
- First assume: fixed congestion window, W segments
- Then dynamic window, modeling slow start

Fixed congestion window (1) [6]

First case:

$WS/R > RTT + S/R$: ACK for first segment in window returns before window's worth of data sent

$$\text{delay} = 2RTT + O/R$$



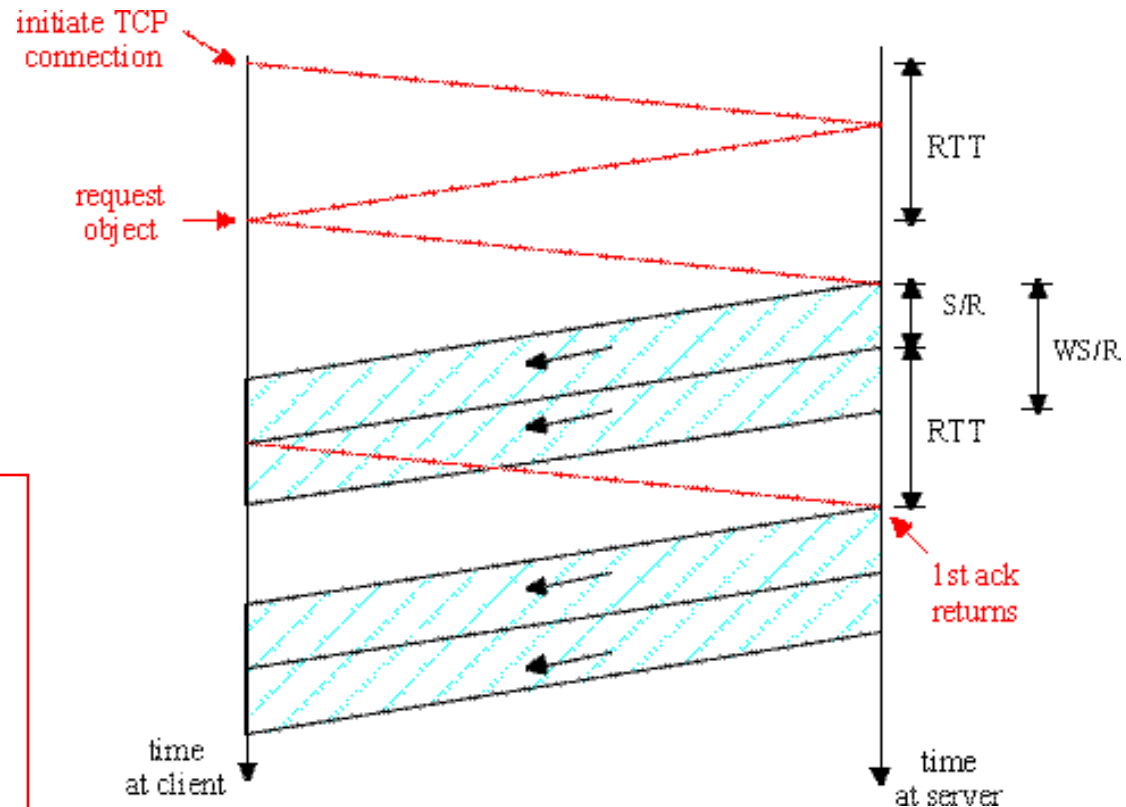
Fixed congestion window (2) [6]

Second case:

- $WS/R < RTT + S/R$: wait for ACK after sending window's worth of data sent

$$\text{delay} = 2RTT + O/R + (K-1)[S/R + RTT - WS/R]$$

Where, $K = \text{Round of}(O/WS)$



References

1. William Stallings, "Data and Computer Communications", Seventh Edition, PHI 2004.
2. Andrew S. Tanenbaum, "Computer Networks" 4th Edition PHI
3. B. A. Fourozan, "TCP/IP Protocol Suite", 3rd Edition, Singapore, McGrawHill, 2004.
4. L. L. Peterson and B. S. Davie, Computer Networks-A System Approach, Elsevier.
5. B. A. Fourozan, "Data Communications and Networking", 4th Edition, Singapore, McGrawHill, 2004.
6. James F. Kurose, Keith W. Ross, "Computer Networking: A Top-Down Approach Featuring the Internet", 3rd Edition /6th Edition , Pearson Education 2009.
7. https://gaia.cs.umass.edu/kurose_ross/ppt.htm
8. PPT available for the respective books

Thank You